ViewSets + Routers

1. What is a ViewSet?

A **ViewSet** is a class that **combines multiple API operations in one place**.

Instead of writing separate views for:

```
GET
POST
PUT
DELETE
```

we can write **one class** that handles all CRUD operations.

DRF provides **ModelViewSet** for this purpose.

---

2. What is ModelViewSet?

ModelViewSet is a class that automatically provides:

Operation	HTTP Method
Get all data	GET
Create data	POST
Get single data	GET
Update data	PUT
Delete data	DELETE

So **CRUD API becomes very small code**.

---

3. What is Router?

Router automatically creates **URL routes for ViewSets**.

DRF provides **DefaultRouter**.

Instead of writing many URLs manually, router **generates them automatically**.

---

4. Example Task

Build **Product API CRUD**.

Fields:

```
id
name
price
description
```

Step 1 — Install DRF

```
pip install djangorestframework
```

Add in **settings.py**

```
INSTALLED_APPS= [
'rest_framework',
]
```

Step 2 — Create Model

models.py

```
fromdjango.dbimportmodels

classProduct(models.Model):

name=models.CharField(max_length=100)
price=models.IntegerField()
```

```
description=models.TextField()

def__str__(self):
returnself.name
```

Run migrations:

```
python manage.py makemigrations
python manage.py migrate
```

Step 3 — Create Serializer

serializers.py

```
fromrest_framework.serializersimportModelSerializer
from.modelsimportProduct

classProductSerializer(ModelSerializer):

classMeta:
model=Product
fields='__all__'
```

Step 4 — Create ViewSet

views.py

```
fromrest_framework.viewsetsimportModelViewSet
from.modelsimportProduct
from.serializersimportProductSerializer


classProductViewSet(ModelViewSet):
```

```
queryset=Product.objects.all()
serializer_class=ProductSerializer
```

This **one class handles full CRUD**.

Step 5 — Create Router

urls.py

```
fromdjango.urlsimportpath,include
fromrest_framework.routersimportDefaultRouter
from.viewsimportProductViewSet

router=DefaultRouter()

router.register('products',ProductViewSet)

urlpatterns= [
path('',include(router.urls)),
]
```

---

6. Auto Generated API Endpoints

Router automatically creates:

Method	Endpoint
GET	/products/
POST	/products/
GET	/products/1/
PUT	/products/1/
DELETE	/products/1/

No need to write each URL manually.

7. Advantages of ViewSets

✔️ Less code

✔️ Automatic CRUD

✔️ Clean structure

✔️ Router creates URLs automatically

8. Simple Flow

```
Model
   ↓
Serializer
   ↓
ModelViewSet
   ↓
Router
   ↓
Auto CRUD APIs
```

9. Student Practical Task

Build **Product API using ModelViewSet**.

Requirements:

Model fields:

```
name
price
description
```

APIs generated:

```
GET      /products/
POST     /products/
GET      /products/<id>/
PUT      /products/<id>/
DELETE   /products/<id>/
```

Step 1 — Create Django Project

Open terminal:

```
django-admin startproject productapi
cd productapi
```

Step 2 — Create App

```
python manage.py startapp products
```

Project structure now:

```
productapi
│
├── productapi
├── products
├── manage.py
```

Step 3 — Install REST Framework

Install:

```
pip install djangorestframework
```

Step 4 — Add Apps in settings.py

Open **productapi/settings.py**

```
INSTALLED_APPS= [

'django.contrib.admin',
'django.contrib.auth',
'django.contrib.contenttypes',
'django.contrib.sessions',
'django.contrib.messages',
'django.contrib.staticfiles',

'rest_framework',
'products',

]
```

Step 5 — Create Product Model

Open **products/models.py**

```
fromdjango.dbimportmodels

classProduct(models.Model):

name=models.CharField(max_length=100)
price=models.IntegerField()
description=models.TextField()

def__str__(self):
returnself.name
```

Fields required by the task:

```
name
price
description
```

Step 6 — Create Database Table

Run migrations:

```
python manage.py makemigrations
python manage.py migrate
```

Now **Product table is created in database**.

Step 7 — Register Model in Admin

Open **products/admin.py**

```
fromdjango.contribimportadmin
from.modelsimportProduct

admin.site.register(Product)
```

Step 8 — Create Serializer

Create file **products/serializers.py**

```
fromrest_framework.serializersimportModelSerializer
from.modelsimportProduct

classProductSerializer(ModelSerializer):

classMeta:
```

```
model=Product
fields='__all__'
```

Serializer converts:

```
Model data → JSON
JSON → Model data
```

Step 9 — Create ViewSet

Open **products/views.py**

```
fromrest_framework.viewsetsimportModelViewSet
from.modelsimportProduct
from.serializersimportProductSerializer


classProductViewSet(ModelViewSet):

queryset=Product.objects.all()
serializer_class=ProductSerializer
```

ModelViewSet automatically provides:

```
list
create
retrieve
update
delete
```

Step 10 — Create Router

Create **products/urls.py**

```
fromdjango.urlsimportpath,include
fromrest_framework.routersimportDefaultRouter
from.viewsimportProductViewSet

router=DefaultRouter()

router.register('products',ProductViewSet)

urlpatterns= [

path('',include(router.urls)),

]
```

Router automatically generates API URLs.

Step 11 — Connect App URLs

Open **productapi/urls.py**

```
fromdjango.contribimportadmin
fromdjango.urlsimportpath,include

urlpatterns= [

path('admin/',admin.site.urls),
path('',include('products.urls')),

]
```

Step 12 — Run Server

```
python manage.py runserver
```

Open browser:

```
http://127.0.0.1:8000/products/
```

You will see **DRF API interface**.

Step 13 — Test API

1️⃣ Get All Products

```
GET /products/
```

Returns:

```
[
 {
  "id":1,
  "name":"Laptop",
  "price":50000,
  "description":"Gaming laptop"
 }
]
```

2️⃣ Add Product

```
POST /products/
```

Body:

```
{
 "name":"Phone",
 "price":20000,
 "description":"Android phone"
}
```

3️⃣ Get Single Product

```
GET /products/1/
```

4️⃣ Update Product

```
PUT /products/1/
```

Body:

```
{
 "name":"Phone",
 "price":22000,
 "description":"Updated phone"
}
```

5️⃣ Delete Product

```
DELETE /products/1/
```

Final Project Structure

```
productapi
│
├── productapi
│    ├── settings.py
│    ├── urls.py
│
├── products
│    ├── models.py
│    ├── serializers.py
│    ├── views.py
│    ├── urls.py
│    ├── admin.py
│
└── manage.py
```

Final Flow

```
Model
   ↓
Serializer
   ↓
ModelViewSet
   ↓
DefaultRouter
   ↓
Auto Generated CRUD APIs
```

10. Very Important Interview Line

```
ModelViewSet automatically provides CRUD operations.
DefaultRouter automatically generates API URLs.
```